

Fine-tuning vs Prompting: Cuándo y Cómo Decidir

Módulo 8 · Ingeniería de Modelos y LLMOps

Framework de decision con cuatro criterios, casos reales, ADR y la evolucion de la decision con el tiempo.

1. Introducción · 2. Qué es · 3. Cómo funciona · 4. Ejemplos reales
5. Errores comunes · 6. Aplicación práctica · 7. Relación con módulos · 8. Resumen ejecutivo

Guía de estudio detallada · +2.000 palabras

Fine-tuning vs Prompting: Cuándo y Cómo Decidir

Uno de los debates más frecuentes en los proyectos de LLM en empresa es: ¿cuándo hacer fine-tuning y cuándo es suficiente el prompting avanzado? La respuesta incorrecta en cualquier dirección tiene consecuencias: hacer fine-tuning cuando el prompting sería suficiente desperdicia semanas de trabajo y presupuesto de cómputo; quedarse con prompting cuando el sistema necesita fine-tuning produce sistemas frágiles que no generalizan bien y consumen presupuesto de tokens innecesariamente.

Este tema proporciona un framework de decisión riguroso basado en el tipo de problema, el volumen de datos disponibles, los requisitos de rendimiento, y las restricciones de coste y latencia. El objetivo no es defender ninguna de las dos opciones sino proporcionar los criterios correctos para elegir la más adecuada para cada situación.

Las diferencias fundamentales entre prompting y fine-tuning

Prompting: configurar el comportamiento sin cambiar el modelo

El prompting (incluyendo zero-shot, few-shot, y prompt engineering avanzado) configura el comportamiento del modelo para cada consulta a través de instrucciones en el contexto. No requiere entrenamiento: los cambios son instantáneos. Los prompts pueden iterarse en horas. El coste es el de las llamadas al API más el de los tokens adicionales en el prompt. La calidad depende de la capacidad del modelo base y del diseño del prompt.

Fine-tuning: incorporar el comportamiento al modelo

El fine-tuning actualiza los pesos del modelo (o de adaptadores LoRA) con ejemplos del comportamiento deseado. Una vez fine-tuneado, el modelo produce el comportamiento correcto sin necesidad de instrucciones extensas en el prompt, reduciendo los tokens por consulta y potencialmente la latencia. El coste es upfront (tiempo de cómputo del entrenamiento) pero se amortiza con volumen de consultas. El riesgo es el catastrophic forgetting si no se gestiona correctamente.

RAG: un tercer camino para problemas de conocimiento

RAG no es una alternativa al fine-tuning o al prompting: es la solución correcta para una clase específica de problemas (acceso a conocimiento actualizado o privado). La decisión no es "RAG vs fine-tuning" sino "¿el problema es de comportamiento/estilo (prompting o fine-tuning) o de conocimiento (RAG)?".

Los criterios para la decisión fine-tuning vs prompting

Criterio 1: ¿El prompting alcanza el rendimiento requerido?

Antes de considerar el fine-tuning, optimiza el prompt completamente y evalúa sobre tu golden dataset con métricas definidas. Si el rendimiento con prompting es >90% del target con el modelo más adecuado (incluyendo few-shot y técnicas avanzadas), el prompting es suficiente. Solo si después de optimización exhaustiva del prompt el rendimiento es insuficiente, el fine-tuning es una opción justificada.

Criterio 2: ¿Tienes suficientes datos de alta calidad?

El fine-tuning requiere un dataset de ejemplos de alta calidad: mínimo 500-1.000 pares (input, output ideal) curados manualmente para casos simples, 2.000-5.000 para comportamientos complejos. Si no tienes esos datos, el fine-tuning producirá un modelo que sobreajusta a los pocos ejemplos disponibles. En ese caso, el prompting few-shot con un pool de ejemplos bien curados es más efectivo.

Criterio 3: ¿El volumen de uso justifica el coste?

El fine-tuning tiene un coste fijo (entrenamiento) y el prompting tiene un coste variable (tokens por consulta). El punto de equilibrio depende del volumen. Si el sistema procesará menos de 10.000 consultas al mes, el prompting suele ser más económico incluso con prompts largos. A 100.000+ consultas al mes, el ahorro de tokens del fine-tuning puede superar el coste de entrenamiento en pocas semanas.

Criterio 4: ¿Hay requisitos de latencia muy estrictos?

Los prompts few-shot con muchos ejemplos aumentan la latencia porque la ventana de contexto es más grande. Si el sistema requiere <500ms de latencia end-to-end y el prompt few-shot óptimo tiene 2.000 tokens de instrucciones y ejemplos, el fine-tuning puede ser la única forma de alcanzar ese target de latencia con un modelo de tamaño comparable.

SECCIÓN 4 · EJEMPLOS REALES

- Caso prompting suficiente (clasificación de sentimiento, e-commerce): el equipo evaluó GPT-4o-mini con un prompt zero-shot y obtuvo 91% de accuracy en su golden dataset de 500 reviews. El target era 90%. Decisión: prompting, sin fine-tuning. Coste mensual: \$50 en tokens. Tiempo de desarrollo: 2 días.
- Caso fine-tuning justificado (extracción de entidades médicas): el equipo necesitaba extraer 15 tipos de entidades médicas específicas de la empresa (diagnósticos, medicamentos, procedimientos con codificación ICD-10 interna). Con prompting, accuracy 74% en el golden dataset de 400 historias clínicas. Target: 92%. Después de fine-tuning de Llama 3.1 8B con 3.000 ejemplos curados por médicos: accuracy 94%. Coste de entrenamiento: \$800. Ahorro mensual en tokens: \$1.200. Amortización: < 1 mes.
- Caso fine-tuning para latencia (asistente en tiempo real): un asistente de atención al cliente necesitaba responder en <800ms. El prompt óptimo tenía 3.000 tokens de instrucciones y ejemplos, produciendo 1.400ms de latencia con GPT-4o-mini. Fine-tuning de Mistral 7B con el mismo set de instrucciones incorporadas: 680ms de latencia, misma calidad, 60% de coste de inferencia.

SECCIÓN 5 · ERRORES Y MALENTENDIDOS COMUNES

Error 1: Fine-tunear antes de optimizar el prompt. El fine-tuning requiere semanas de trabajo y presupuesto de cómputo. El prompting requiere horas. Siempre optimiza el prompt primero con evaluación sistemática sobre el golden dataset. Si el prompting alcanza el rendimiento requerido, el fine-tuning es innecesario.

Error 2: Fine-tunear para añadir conocimiento cuando RAG sería la solución. Si el problema es que el modelo no sabe sobre los productos, documentos, o procesos de tu empresa, fine-tunear el modelo con esa información es menos eficiente que usar RAG. El conocimiento en un modelo fine-tuneado es estático; en RAG, se actualiza sin re-entrenar.

Error 3: Evaluar el fine-tuning solo con los datos de entrenamiento. El fine-tuning siempre mejora el rendimiento en el dataset de entrenamiento (puede que por sobreajuste). La evaluación correcta es con un hold-out test set representativo y el golden dataset de casos reales. Evalúa también el rendimiento en tareas generales (MMLU subset) para detectar catastrophic forgetting.

SECCIÓN 6 · APLICACIÓN PRÁCTICA

El proceso de decisión recomendado: paso 1, define los requisitos (rendimiento objetivo, latencia máxima, coste máximo mensual, volumen esperado). Paso 2, construye el golden dataset (50-200 casos con outputs ideales). Paso 3, evalúa el mejor prompt que puedas diseñar en el golden dataset. Si supera el objetivo → go con prompting. Paso 4, si prompting es insuficiente, evalúa si el problema es de conocimiento (→ RAG) o de comportamiento (→ fine-tuning). Paso 5, si fine-tuning, estima el coste (training + inferencia) y el ROI. Si el ROI es positivo y tienes suficientes datos → fine-tunear.

Para documentar la decisión: crea un decision record (ADR - Architecture Decision Record) con: el problema que resuelves, las opciones evaluadas (prompting, RAG, fine-tuning), los datos de evaluación de cada opción (métricas en el golden dataset, coste estimado, latencia medida), la decisión tomada y los criterios que la motivaron. Este documento es valioso cuando el equipo cambia o cuando se revisa la decisión en el futuro.

SECCIÓN 7 · RELACIÓN CON OTROS MÓDULOS

- M4-T6 (Fine-tuning): la implementación técnica del fine-tuning con LoRA, QLoRA y TRL.
- M5-T3 (Pretraining vs fine-tuning): el contexto teórico de por qué y cómo funciona el fine-tuning.
- M7-T1 (RAG): la alternativa al fine-tuning para problemas de conocimiento.

SECCIÓN 8 · RESUMEN EJECUTIVO

El framework de decisión tiene cuatro criterios: ¿el prompting alcanza el rendimiento requerido? (evalúa primero), ¿tienes 500-5.000 pares de alta calidad para fine-tuning?, ¿el volumen justifica el coste fijo del entrenamiento vs el coste variable de tokens?, ¿hay requisitos de latencia muy estrictos que prompts largos no pueden cumplir? Si el problema es de conocimiento → RAG (no fine-tuning). Si fine-tuning: mínimo 500-1.000 ejemplos curados, evalúa con hold-out test set y golden dataset, chequea catastrophic forgetting. El prompting primero es la regla de oro: semanas de fine-tuning vs horas de

prompt engineering.